

Deep Dive: Migrating the Intake Flask App to Vercel

Status: deep dive / research only — no code changes. Related: the platform-unification spike (option 2.4), now the leading direction. Prepared: 2026-07-30

1. Why this deep dive

The platform-unification spike flagged migrating the Flask intake app onto Vercel — eliminating Render entirely — as attractive but blocked by two things needing more than a paragraph: Vercel Functions' request-body limit against the app's 10MB upload requirement, and Flask-Limiter's in-memory rate-limit storage not working across serverless instances. This document goes deep on both, plus the rest of what a real migration touches.

2. The headline fact, confirmed today

Vercel Functions still cap request/response bodies at 4.5MB. This is stated plainly on Vercel's current limits page (vercel.com/docs/functions/limitations, last updated 2026-07-01, fetched directly for this document):

> "The maximum payload size for the request body or the response body of a > Vercel Function is 4.5 MB. If a Vercel Function receives a payload in > excess of the limit it will return an error 413: > FUNCTION_PAYLOAD_TOO_LARGE."

This is worth stating explicitly because initial research surfaced a conflicting claim (a "100MB" figure from an internal knowledge note) that turned out to be stale or simply wrong — checked directly against the live docs to be sure before writing anything else here. 4.5MB is the real, current ceiling, and the app's `MAX_CONTENT_LENGTH = 10 * 1024 * 1024` exceeds it outright. A straight lift-and-shift of `POST /submit` as multipart form data will not work unmodified.

One adjacent fact worth clearing up now, since it initially looked like it might offer an escape hatch: Resend's own attachment limit is 40MB per email after base64 encoding (confirmed from Resend's API reference — "max 40MB per email, after Base64 encoding of the attachments"). Resend was never the bottleneck; the wall is purely Vercel's Function body limit on the way in, not the outbound email on the way out.

3. Project structure & config — the easy part

Confirmed from vercel.com/docs/frameworks/backend/flask (updated 2026-07-06):

- No file relocation needed. Vercel auto-detects a Flask instance named

app at `app.py` (among other conventional names) at the repo root — the current layout works as-is.

- `vercel.json` needs only a minimal function config, e.g.:

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "functions": { "app.py": { "maxDuration": 60 } }
}
```

- Static assets must move. Vercel's docs are explicit: `Flask's`

`app.static_folder` should not be used for static files on Vercel. Use the `public/**` directory instead. `static/form.js` and any CSS need to move to a top-level `public/` directory, and `url_for('static', ...)` calls in `templates/index.html` need to become plain paths. This is a small, mechanical change — a handful of `<script src>`/`<link href>` edits — but a real one, not optional.

- `templates/` is unaffected. Nothing in Vercel's docs suggests moving

Jinja2 templates — they're read from disk by Python at request time, not CDN-served static content, so `render_template()` keeps working as-is. (Inferred from the docs' silence on this point, not stated outright — worth a smoke test, not a leap of faith.)

- Bundle size is a non-issue. Python functions get a 500MB uncompressed

limit; `reportlab` is pure Python with no heavy binary dependencies, and the app's full dependency set (Flask, gunicorn, Flask-Limiter, Flask-WTF, `reportlab`, the Resend SDK) is tens of MB at most.

- `vc init flask` scaffolds from Vercel's own example repo

(github.com/vercel/vercel/tree/main/examples/flask) if a fresh reference layout is useful; `vercel dev` runs it locally.

4. Solving the upload-size problem: three options

Option A — lower the accepted upload size below 4.5MB

The simplest possible fix: shrink `MAX_CONTENT_LENGTH` from 10MB to something safely under 4.5MB (accounting for the other text fields sharing the same multipart body — a cap around 3.5–4MB total leaves headroom). Zero new infrastructure, zero new client-side JavaScript, zero new runtime to maintain. The tradeoff is real, though: it narrows what a client can actually upload — a multi-page scanned CV or a bulky LinkedIn export PDF could plausibly exceed a 4MB cap where they don't today.

Checked directly against real submission data, and the answer is clear. The app itself only ever logged upload count, not size (S-15's app. logger call records `uploads=%d`, nothing byte-related), so there's no systematic server-side log of this — but the actual files from roughly a dozen past client engagements are still retained locally in `Clients/`. Isolating genuine uploads (CVs, JDs, LinkedIn exports) from generated outputs (transition plans, intake PDFs, the consultant's own `generate_*.py` scripts):

- The single largest real upload observed is Christine Karoki's CV at

455,887 bytes (~445KB) — under 10% of the current 10MB ceiling, and under 15% of even a conservative 4MB cap.

- Summing every plausible upload-like file *within a single client's

folder* (i.e. the worst-case combined request-body size, since Vercel's 4.5MB limit applies to the whole body, not per file) tops out at ~654KB (Jackie's folder — and that figure likely overcounts, since it mixes what look like consultant-produced CV rewrites in with the original). Every other client's combined total is well under 220KB.

- The only files anywhere near multi-MB size live in an unrelated personal

folder (WhatsApp chat exports, a marketing PDF) that has nothing to do with the intake form's upload fields and was almost certainly never actually submitted through it — excluded from the analysis above.

So across every real submission on record, nothing comes remotely close to even a 4MB cap, let alone 4.5MB. Option A is clearly supported by actual usage and is the recommended starting point — with the

caveat that this is retained-files evidence from ~12 past engagements, not a guarantee against a future outlier (a scanned multi-page CV, a large portfolio PDF). Worth adding actual byte-size logging to `app.logger` alongside the existing `uploads=%d` count regardless of which option is chosen, so this becomes an ongoing measured fact rather than a one-time spot check.

Option B — Vercel Blob client uploads, with a small Node token-issuing function

Vercel's own guide for bypassing the 4.5MB limit (vercel.com/kb/guide/how-to-bypass-vercel-body-size-limit-serverless-functions, fetched directly) recommends exactly this: have the browser upload large files directly to Vercel Blob storage, bypassing the Function as a proxy entirely, rather than routing bytes through `/submit` at all.

The mechanics, end to end:

1. The browser, per file input, calls `upload()` from `@vercel/blob/client`

instead of building one `FormData` for everything as today.

2. Each `upload()` call needs a short-lived authorization token, minted by

a server-side handler implementing the `handleUpload()` token-exchange protocol. This half of the flow is JS/TS-only — `@vercel/blob's handleUpload()` helper has no Python equivalent, and no public, versioned raw HTTP contract was found that a Flask endpoint could reimplement with confidence. The realistic path is a second, small Node.js function living alongside the Flask app in the same Vercel project (e.g. `api/blob-token.js`) whose only job is issuing these tokens — Vercel projects can mix runtimes per-function, so this is workable, just a second language to maintain for one endpoint.

3. Once uploaded, each file yields a Blob URL. The browser submits those

URLs plus the remaining text fields to `/submit` (well under 4.5MB now, since it's just strings) instead of raw file bytes.

4. `/submit` fetches the bytes back from each Blob URL server-side before

handing them to Resend as attachments (Resend's 40MB ceiling comfortably covers this).

5. Vercel Blob has no automatic expiry — uploaded files persist until

explicitly deleted. The current app's temp-file cleanup pattern needs an equivalent: call `Blob's del()` after the PDF is built and emailed, or files accumulate indefinitely toward the plan's storage cap.

Multiple files per submission (CV, job description, LinkedIn export, portfolio docs, additional documents) means multiple concurrent `upload()` calls client-side (`Promise.all()`), with its own partial-failure/retry handling — this is genuinely more client JavaScript than today's single `fetch('/submit', {body: new FormData(this)})` call, roughly 40–60 lines of new JS plus error handling, not a drop-in swap.

Pricing at this consultant's volume is a non-concern either way — Blob's Hobby-tier allowances (referenced in Vercel's pricing examples: 5GB storage, 100K simple operations, 10K advanced operations) are far beyond a handful of submissions a day, and client uploads themselves carry no data-transfer charge.

Option C — reimplement the Blob token protocol in pure Python

Explicitly not recommended. There is no confirmed, stable, public HTTP contract for the token-exchange step to reimplement — doing this would mean reverse-engineering the `@vercel/blob` npm package's internals, which is fragile and likely to break silently on a future SDK update with no changelog entry

pointing at why uploads suddenly stopped authorizing.

Recommendation

Option A, confirmed. Real submission data (above) shows every past upload, and every plausible worst-case combined submission, landing well under even a conservative 4MB cap — this is a same-day change with no new infrastructure. Option B stays on the shelf as a fallback only if a future client's submission genuinely needs more than that (the officially-supported path, just a real chunk of new work — a second runtime, new client JS, an explicit cleanup step). Option C stays ruled out.

5. Rate limiting: moving off in-memory storage

Flask-Limiter's `storage_uri` expects a standard Redis-protocol connection (`redis:///rediss://`) — confirmed from its own docs, which state extra `storage_options` pass straight to `redis.Redis.from_url()`. This matters because Upstash's own Flask quickstart uses its REST-based Python client (`upstash_redis.Redis.from_env()`), which is HTTP-based and is not what Flask-Limiter needs — the two don't compose directly.

The practical path: Upstash Redis databases (independent of how they're provisioned) are generally understood to expose both a REST endpoint and a standard `rediss://` TCP endpoint for exactly this compatibility reason, but this deep dive could not confirm from Vercel-specific docs alone that the Vercel Marketplace's Upstash integration surfaces that TCP connection string alongside the REST env vars it's documented to inject — this needs confirming directly in the dashboard at provisioning time, not assumed from general Upstash knowledge.

Once a `rediss://` string is in hand, point `storage_uri` at it with an explicit connection pool held at module scope — Fluid Compute reuses warm instances across requests (unlike classic one-request-per-container serverless), so a pooled connection can survive across invocations rather than reconnecting every request, which matters given Vercel Functions share a hard 1,024 file-descriptor ceiling across all concurrent executions on an instance.

6. Runtime fit for this specific workload

Confirmed current limits (vercel.com/docs/functions/limitations):

Limit	Hobby	Pro/Enterprise
Max memory	2GB / 1 vCPU	2GB default, 4GB / 2 vCPU max
Max duration	300s default and max	300s default, 800s max, 1800s extended (beta)
Bundle size (Python)	500MB uncompressed	same, 5GB with Large Functions (beta)
File descriptors	1,024 shared across concurrent executions	same

This workload's shape actually fits Vercel's pricing model well: the `reportlab` PDF build is real, billable active-CPU time, but the outbound HTTPS call to Resend is I/O wait, which is not billed as active CPU (only provisioned-memory time accrues during that wait, at a lower rate). The 300s default timeout leaves enormous headroom over what this synchronous build-and-send request actually needs.

7. CSRF in a serverless request model

Flask-WTF's CSRF token is signed with `SECRET_KEY` and validated via Flask's default client-side signed cookie session backend (not a server-side session store) — so there's no server-side session state for a stateless Function model to break. The one real gotcha: `SECRET_KEY` (and `WTF_CSRF_SECRET_KEY` if set separately) must be a fixed environment variable, not generated randomly at process startup — if different warm instances each minted their own random key, a token issued by the instance serving `GET /` could fail validation on a different instance handling the subsequent `POST /submit`, surfacing as intermittent, hard-to-reproduce CSRF failures under scaling. This is almost certainly already correct today (the same failure mode would already show up under Render's own scaling if it weren't) — worth an explicit "verify, don't assume" check during migration, not a rewrite.

8. What this means for domain unification

Once the Flask app is itself a Vercel project, unifying it with the Next.js marketing site under one domain becomes a same-account Vercel-to-Vercel rewrite:

```
{
  "rewrites": [
    { "source": "/apply/:match*", "destination": "https://career-transition-intake.vercel.app/:match*" },
    { "source": "/:match*", "destination": "https://your-nextjs-marketing-site.vercel.app/:match*" }
  ]
}
```

Vercel's own multi-app product (@vercel/microfrontends) explicitly does not support Python/Flask — its docs list only JS frameworks (Next.js, SvelteKit, React Router, Vite) — and even its own fallback guidance for a non-Vercel-native backend is "create a Vercel project that rewrites to the external host," i.e. the same plain rewrite mechanism, not a different one. So this isn't a fundamentally different or more "native" routing path than the external-origin rewrite investigated for Render — it's the same proxy mechanism, just with both origins now inside Vercel's own edge network, which plausibly (not confirmed with benchmark data) avoids a cross-cloud TLS/DNS hop.

The genuine payoff of this migration, independent of domain unification, is that the loading page can be deleted outright, not migrated — Fluid Compute's Python cold starts are sub-second to low-seconds, removing the entire reason that page (polling `/_health` to mask Render's ~60s free-tier wake-up) exists.

9. Migration plan and effort estimate

1. **Lower `MAX_CONTENT_LENGTH` to ~4MB (Option A, confirmed by real submission data in §4)** — no new infrastructure, done in an hour.
2. Add `vercel.json`; no relocation needed for `app.py`.
3. Move static assets to `public/`, update template references (~half a day, mostly rewriting a handful of tags and testing).
4. Provision Upstash Redis via the Vercel Marketplace, confirm the actual connection-string variant at provisioning time, point Flask-Limiter's `storage_uri` at it with a pooled connection (~half a day).
5. Verify `SECRET_KEY`/`WTF_CSRF_SECRET_KEY` are fixed env vars (~30 minutes, likely already correct).

6. Deploy to a Vercel preview; smoke-test `/submit` end to end (multi-file

upload, PDF build, Resend send, `/_health` if kept), including triggering the rate limit deliberately to confirm Redis-backed limiting actually enforces across concurrent invocations — the entire point of the move off `MemoryStorage`.

7. Set up the domain-split rewrite in the Next.js project, cut DNS over, decommission Render — delete the loading page rather than port it.

Effort estimate for a solo Flask dev: roughly 2-3 working days, now that Option A is confirmed viable and Option B's multi-day Blob integration isn't on the critical path. Sequence this migration either just before or just after the parallel payment-gating rework of `/submit` (from the payment-integration spike), not concurrently — both touch the same route and rebasing both at once invites avoidable mistakes.

Single riskiest step: provisioning Upstash Redis and confirming which connection-string variant it actually exposes — everything else in this plan is either already confirmed from primary docs or a mechanical config change. Option B's Node token-issuing function is no longer on the critical path, but stays documented above as the fallback if a future submission ever exceeds the ~4MB cap.

10. Open questions

- Does the Vercel Marketplace's Upstash integration expose a `redis://` TCP

connection string, or only the REST vars? Confirm at provisioning time, not from general Upstash product knowledge.

- Is `SECRET_KEY` already a fixed Render env var today (almost certainly yes, but verify before relying on it post-migration)?

- Land this migration before or after the payment-gating rework of `/submit`? Recommend not concurrently, either order otherwise fine.

Sources

- [Vercel — Functions Limits](#)
- [Vercel — How to bypass the 4.5MB body size limit](#)
- [Vercel — Deploy a Flask app](#)
- [Vercel — Python Runtime](#)
- [Vercel — Client Uploads with Vercel Blob](#)
- [Vercel — Blob overview](#)
- [Vercel — Blob usage and pricing](#)
- [Vercel — Redis on Vercel](#)
- [Vercel — How can I serve multiple projects under a single domain?](#)
- [Flask-Limiter documentation](#)
- [Upstash — Flask quickstart](#)
- [Resend — Send email API reference](#)